CS 70 离散数学 和 概率论
笔记 11
春季 2026 Course Notes
Self-Reference 和 Computability
In this lecture we will explore the deep connection between
proofs 和 computation. At the heart of this

connectionisthenotionofself-reference,andithasfar-reachingconsequencesforthelimitsofcomputation

(theHaltingProblem)andthefoundationsoflogicinmathematics(Gödel’sIncompletenessTheorem).
1 The Liar’s Paradox
回忆 propositions are statements that are either 真 或 假. We saw
in an earlier lecture that some
statements are 非 well defined 或 too imprecise to be called
propositions. 但是 here is a statement 即
problematicformoresubtlereasons:
“AllCretansareliars.”
所以 said a Cretan in antiquity, 因此 giving rise to the
所以-called liar’s paradox 其中 has amused 和 con-
foundedpeopleoverthecenturies. Why?
Becauseifthestatementaboveistrue,thentheCretanwaslying,
whichimpliesthestatementisfalse.
Butactuallytheabovestatementisn’treallyaparadox;itsimplyyields

acontradictionifweassumeitistrue,butifitisfalsethenthereisnoproblem.
Atrueformulationofthisparadoxisthefollowingstatement:
“Thisstatementisfalse.”
Is the statement above 真? 如果 the statement is 真, 那么 what it
asserts must be 真; 即 that it is
假. 但是 如果 it is 假, 那么 it must be 真. 所以 it really is a
paradox, 和 we see that it arises because of
theself-referentialnatureofthestatement. Aroundacenturyago,
thisparadoxfounditselfatthecenterof
foundationalquestionsaboutmathematicsandcomputation.
We will now study how this paradox relates to computation.
Before doing 所以, 令 us 考虑 another
manifestation of the paradox, created by the great logician
Bertrand Russell. In a village with just one
barber, every man keeps himself clean-shaven. Some of the
men shave themselves, 当...时 others go to the
barber. Thebarberproclaims:
“Ishaveallandonlythosemenwhodonotshavethemselves.”
Itseemsreasonablethentoaskthequestion:
Doesthebarbershavehimself? Thinkingmorecarefullyabout
the 问题 though, we see that, assuming that the barber’s
statement is 真, we are presented with the
same self-referential paradox: a logically impossible
scenario. 如果 the barber does 非 shave himself, 那么
accordingtowhatheannounced,heshaveshimself.
Ifthebarberdoesshavehimself,thenaccordingtohis
statementhedoesnotshavehimself!

2 Self-Replicating Programs
Canweuseself-referencetodesignaprogramthatoutputsitself?
Toillustratetheidea,letusconsiderhow
wecandothisifwecouldwritetheprograminEnglish.
Considerthefollowinginstruction:
Print out the following sentence:
“Print out the following sentence:”

Ifweexecutetheinstructionabove(interpretingitasaprogram),thenwewillgetthefollowingoutput:
Printoutthefollowingsentence:
Clearly this is 非 the same as the original instruction
above, 其中 consists of two lines. We can try to
modifytheinstructionasfollows:
Print out the following sentence twice:
“Print out the following sentence twice:”

Executingthismodifiedinstructionyieldstheoutputwhichnowconsistsoftwolines:
Printoutthefollowingsentencetwice:
Printoutthefollowingsentencetwice:

Thisalmostworks,exceptthatwearemissingthequotesinthesecondline.
Wecanfixitbymodifyingthe
instructionasfollows:
Print out the following sentence twice, the second time in
quotes:
“Print out the following sentence twice, the second time in
quotes:”

Thenweseethatwhenweexecutethisinstruction,wegetexactlythesameoutputastheinstructionitself:
Printoutthefollowingsentencetwice,thesecondtimeinquotes:
“Printoutthefollowingsentencetwice,thesecondtimeinquotes:”
QuinesandtheRecursionTheorem

Intheabovesectionwehaveseenhowtowriteaself-replicatingprograminEnglish.
Butcanwedothisin
arealprogramminglanguage?
Ingeneral,aprogramthatprintsitselfiscalledaquine,1
anditturnsoutwe
canalwayswritequinesinanyprogramminglanguage.
Asanotherexample,considerthefollowingpseudocode:
(Quine “s”)
(s “s”)

Thepseudocodeabovedefinesaprogramquinethattakesastringsasinput,
andoutputs(s “s”), 其中
means we run the string s (now interpreted as a program) on
itself. Now 考虑 executing the program
Quinewithinput“Quine”:
(Quine “Quine”)

1Quineisnamedafterthephilosopherandlogician WillardVanOrmanQuine,aspopularizedinthebook “Gödel,Escher,Bach:
AnEternalGoldenBraid” byDouglasHofstadter.

Bydefinition,thiswilloutput
(Quine“Quine”)
whichisthesameastheinstructionthatweexecuted!
Thisisasimpleexample,buthowdoweconstructquinesingeneral?
Theanswerisgivenbytherecursion
定理. The 递归 定理 states that 给定 any program P(x,y), we can
always convert it to another

programQ(x)suchthatQ(x)=P(x,Q),i.e.,QbehavesexactlyasPwouldifitssecondinputisthedescrip-
tion of the program Q. In this sense we can think of Q as a
“self-aware” version of P, 因为 Q essentially
hasaccesstoitsowndescription.
3 The Halting 题目

Are there tasks that a computer cannot perform? 对于 例子, we
would like to ask the following basic
questionwhencompilingaprogram: doesitgointoaninfiniteloop?
In1936,AlanTuringshowedthatthere
isnoprogramthatcanperformthistest.
Theproofofthisremarkablefactisveryelegantandcombinestwo
ingredients: self-reference (as in the liar's paradox), 和
the fact that we cannot separate programs from
data. In computers, a program is represented by a string of
bits just as integers, characters, 和 other data
are. Theonlydifferenceisinhowthestringofbitsisinterpreted.
WewillnowexaminetheHaltingProblem.
Giventhedescriptionofaprogramanditsinput,wewouldlike
to know 如果 the program ever halts when it is executed on the
给定 input. 换句话说, we would like to
writeaprogramTestHaltthatbehavesasfollows:
(cid:40)
"yes", ifprogramPhaltsoninputx
TestHalt(P,x)=
"no", ifprogramPloopsoninputx
Whycan't suchaprogramexist?
First,letususethefactthataprogramisjustabitstring,soitcanbeinput
asdata.
ThismeansthatitisperfectlyvalidtoconsiderthebehaviorofTestHalt(P,P),whichwilloutput
"yes"ifPhaltsonP,和"no"ifPloopsforeveronP.
WenowprovethatsuchaprogramcannotexistA.
定理 11.1. The Halting 题目 is uncomputable; i.e., there does 非
exist a computer program
TestHaltwiththebehaviorspecifiedaboveonallinputs(P,x).
证明. 假设 对于 矛盾 the existence of the program TestHalt. 那么 we
can use it to construct
thefollowingprogram:
Turing(P)
如果 TestHalt(P, P) = "yes" 那么 loop forever
else halt
所以 如果 the program P when 给定 P as input halts, 那么 Turing(P)
loops forever; otherwise, Turing(P)
halts.
AssumingwehavetheprogramTestHalt,wecaneasilyuseitasasubroutineintheaboveprogram
Turing.
Now 令 us look at the behavior of Turing(Turing). 存在 two
cases: either it halts, 或 it does 非.
如果 Turing(Turing) halts, 那么 it must be the case that
TestHalt(Turing,Turing) returned "no." 但是
by 定义 of TestHalt, that would mean that Turing(Turing)
should 非 have halted. In the second

case,ifTuring(Turing)doesnothalt,thenitmustbethecasethatTestHalt(Turing,
Turing)returned
"yes,"whichwouldmeanthatTuring(Turing)shouldhavehalted.

In both cases, we arrive at a 矛盾 其中 must mean that our
initial assumption, 即 that the
program TestHalt exists, was wrong. 因此, TestHalt cannot
exist, 所以 it is impossible 对于 a program to
checkifanygeneralprogramhalts!
What 证明 technique did we use? This was actually a 证明 by
diagonalization, the same technique that
we used in the previous lecture to 证明 that the 实数 numbers
are uncountable. Why? 因为 the 集合 of all

computerprogramsiscountable(theyare,afterall,justfinite-lengthstringsoversomealphabet,andtheset

ofallfinite-lengthstringsiscountable),wecanenumerateallprogramsasfollows(whereP
representsthe
i
ith program):
P P P P P ...
0 1 2 3 4
P 0 H L H L H ...
P 1 L L H L L ...
P 2 H H H H L ...
P 3 H H H H H ...
.
.
.
The (i,j)th entry in the 表 above is H 如果 program P halts on
input P, 和 L (对于 “Loops”) 如果 it does 非
i j
halt. Now 如果 the program Turing exists it must occur
somewhere on our list of programs, say as P . 但是
n
this cannot be, 因为 如果 the nth entry in the diagonal is H,
meaning that P halts on P , 那么 by its 定义
n n
Turing loops on P ; 和 如果 the entry is L, 那么 by 定义 Turing
halts on P . 因此 the behavior of
n n
Turing is different from that of P , 和 因此 Turing does 非
appear on our list. 因为 the list contains
n
all possible programs, we must conclude that the program
Turing does 非 exist. 和 因为 Turing is
constructed by a simple modification of TestHalt, we can
conclude that TestHalt does 非 exist either.
HencetheHaltingProblemcannotbesolved.
In fact, 存在 many more questions we would like to 答案 about
programs 但是 cannot. 对于 例子,
we cannot know 如果 a program ever outputs anything 或 如果 it
ever executes a specific line. We also cannot
checkiftwoprogramsproducethesameoutput.
Andwecannotchecktoseeifagivenprogramisavirus.

TheseissuesareexploredingreaterdetailintheadvancedcourseCS172(ComputabilityandComplexity).
TheEasyHaltingProblem
As noted above, the key idea in establishing the
uncomputability of the Halting 题目 is self-reference:
给定 a program P, we ran into trouble when deciding whether
P(P) halts. 但是 in practice, how often do
wewanttoexecuteaprogramwithitsowndescriptionasinput?
Isitpossiblethatifwedisallowthiskind
ofself-reference,wecansolvetheHaltingProblem?
对于 例子, 给定 a program P, what 如果 we ask instead the 问题: “Does
P halt on input 0?” This

lookseasierthantheHaltingProblem(hencethenameEasyHaltingProblem),sinceweonlyneedtocheck

whetherPhaltsonaspecificinput0,insteadofanarbitrarygiveninput(suchasPitself).
然而,itturns

out this seemingly easier 题目 is still uncomputable! We 证明
this claim by showing that 如果 we could
solve the Easy Halting 题目, 那么 we could also solve the
Halting 题目 itself; 因为 we know the

HaltingProblemisuncomputable,thisimpliestheEasyHaltingProblemmustalsobeuncomputable.

Specifically,supposewehaveaprogramTestEasyHaltthatsolvestheEasyHaltingProblem:
(cid:40)
“yes”, ifprogramPhaltsoninput0
TestEasyHalt(P)=
“no”, ifprogramPloopsoninput0
CS70,Spring2026,Note11 4

ThenwecanuseTestEasyHaltasasubroutineinthefollowingalgorithmthatsolvestheHaltingProblem:
Halt(P, x)
construct a program P’ that, on input 0, returns P(x)
return TestEasyHalt(P’)
ThealgorithmHaltconstructsanotherprogramP′,
whichdependsonboththeoriginalprogramPandthe
original input x, 满足 when we call P′(0) we return P(x). Such
a program P′ can be constructed very
simplyasfollows:
P’(y)
return P(x)
Thatis,thenewprogramP′ ignoresitsinputyandalwaysreturnsP(x).
(NotethatthedescriptionsofPand
ofxare“hard-wired”intoP′.)
ThenweseethatP′(0)haltsifandonlyifP(x)halts. 因此, ifwehave
such a program TestEasyHalt, 那么 Halt will correctly solve
the Halting 题目. 因为 we know there

cannotbesuchaprogramHalt,weconcludeTestEasyHaltdoesnotexisteither.
The technique that we use here is called a 归约. Here we are
reducing one 题目 “Does P halt on
x?” toanotherproblem“DoesP′
halton0?”,inthesensethatifweknowhowtosolvethesecondproblem,
那么 we can use that knowledge to construct an 答案 对于 the first
题目. This 蕴含 that the second

problemisactuallyasdifficultasthefirst,despitetheapparentlysimplerdescriptionofthesecondproblem.
4 Godel’s Incompleteness 定理
In 1900, the great mathematician David Hilbert posed the
following two questions about the foundation of
logicinmathematics:
1. Isarithmeticconsistent?
2. Isarithmeticcomplete?

Tounderstandthequestionsabove,werecallthatmathematicsisaformalsystembasedonalistofaxioms

(forexample,Peano’saxiomsofthenaturalnumbers,axiomofchoice,etc.)
togetherwithrulesofinference.

Theaxiomsprovidetheinitiallistoftruestatementsinoursystem,andwecanapplytherulesofinference

toproveothertruestatements,whichwecanagainusetoproveotherstatements,andsoon.
The first 问题 above asks whether it is possible to 证明 both a
命题 P 和 its negation ¬P. 如果
this is the case, 那么 we say that arithmetic is inconsistent;
otherwise, we say arithmetic is consistent. 如果

arithmetic is inconsistent, meaning 存在 假 statements that can
be proved, 那么 the entire arithmetic
system will collapse because from a 假 statement we can
deduce anything, 所以 every statement in our
systemwillbevacuouslytrue.

Thesecondquestionaboveaskswhethereverytruestatementinarithmeticcanbeproved.
Ifthisisthecase,
thenwesaythatarithmeticiscomplete.
Wenotethatgivenastatement, whichiseithertrueorfalse, itcan
beverydifficulttoprovewhichoneitis.
Asareal-worldexample,considerthefollowingstatement,其中
isknownasFermat' sLastTheorem:
(n≥3)¬(x,y,z∈Z+)(xn+yn=zn).
This 定理 was first stated by Pierre de Fermat in 1637,2 但是 it
had eluded proofs 对于 centuries 直到 it

2Alongwiththefamousnote:“Ihavediscoveredatrulymarvelousproofofthis,whichthismarginistoonarrowtocontain.”

wasfinallyprovedbyAndrewWilesin1994.
Otherfamousmathematicalstatements,suchastheRiemann
假设,remainopentothisday.
In 1928, Hilbert formally posed the questions above as the
Entscheidungsproblem. Most people believed

thattheanswerwouldbe“yes,”sinceideallyarithmeticshouldbebothconsistentandcomplete.
然而,
in 1930 Kurt Gödel proved that the 答案 is in fact “no”: Any
formal system 即 sufficiently rich
to formalize arithmetic is either inconsistent (存在 假
statements that can be proved) 或 incomplete
(therearetruestatementsthatcannotbeproved).
Gödelprovedhisresultbyexploitingthedeepconnection
betweenproofsandarithmetic.
ActuallyGödel' stheoremalsoembodiesadeepconnectionbetweenproofs
和 computation, 其中 was illuminated after Turing formalized
the 定义 of computation in 1936 via
thenotionofTuringmachinesandcomputability.
Intherestofthisnote,
wewillfirstsketchtheessenceofGödel' sproof,
andthenwewilloutlineaneasier
proofofthetheoremusingwhatweknowabouttheHaltingProblem.
SketchofGödel' sProof
假设 我们有 a formal system F, 其中 consists of a list of axioms 和
rules of inference, 和 假设
F
issufficientlyexpressivethatwecanuseittoexpressallofarithmetic.
Nowsupposewecanwritethefollowingstatement:
S(F)=“ThisstatementisnotprovableinF.”
Oncewehavethisstatement,therearetwopossibilities:
1. Case 1: S(F) is provable. 那么 the statement S(F) is 真, 但是
by inspecting the content of the
statement itself, we seethat this impliesS(F)should 非 be
provable. 因此, F is inconsistent inthis
case.
2. Case 2: S(F) is 非 provable. 通过构造, this means the
statement S(F) is 真. 因此, F is

incompleteinthiscase,sincethereisatruestatement(即,S(F))thatisnotprovable.

Tocompletetheproof,itnowsufficestoconstructsuchastatementS(F).
ThisisthedifficultpartofGödel’s
证明, 其中 requires a clever encoding (a 所以-called “Gödel
numbering”) of symbols 和 propositions as
naturalnumbers.
ProofviatheHaltingProblem

LetusnowseehowwecanproveGödel’sresultbyreductiontotheHaltingProblem.
Hereweproceedby
矛盾:
Supposearithmeticisbothconsistentandcomplete;wewillusethisassumptiontosolvethe
HaltingProblem,whichwehaveseenisimpossible.

RecallthatintheHaltingProblemwewanttodecidewhetheragivenprogramPhaltsonagiveninputx.
对于
fixedPandx,letS denotetheproposition“Phaltsoninputx.”
Thekeyobservationisthatthisproposition
P,x
canbephrasedasastatementinarithmetic. TheformofthestatementS
willbe
P,x
  z(zencodesavalidhaltingexecutionsequenceofPoninputx).
虽然 the details require some work, your programming intuition
should hopefully convince you that

suchastatementcanbewritten,inafairlymechanicalway,usingonlythelanguageofstandardarithmetic,
withtheusualoperators,connectivesandquantifiers:
basicallythestatementjusthastocheck,stepbystep,
CS70,Spring2026,Note11 6

that the string z (encoded as a very long 整数 in binary)
lists out the 序列 of states that a computer
wouldgothroughwhenrunningprogramPoninputx.
Now 令 us 假设, 对于 矛盾, that arithmetic is both consistent 和
完全的. This means that, 对于
any(P,x),thestatementS
iseithertrueorfalse,andthattheremustexistaproofinarithmeticofeitherS
P,x P,x
oritsnegation,¬S (andnotboth).
ButnowrecallthataproofissimplyafinitebinarystringP,x. 因此,

thereareonlycountablymanypossibleproofs,sowecanenumeratethemonebyoneandsearchforaproof
ofS 或¬S . Thefollowingprogramperformsthistask:
P,x P,x
Search(P, x)
对于 every 证明 q:
如果 q is a 证明 of S 那么 output "yes"
P,x
如果 q is a 证明 of ¬S 那么 output "no"
P,x

TheprogramSearchtakesasinputtheprogramP,andproceedstocheckeverypossibleproofuntilitfinds
either one that proves S , 或 one that proves ¬S . 根据假设, we
know that one (和 only one) of
P,x P,x
these proofs always exists, 所以 the program Search will
terminate in 有限的 time, 和 it will correctly solve
the Halting 题目. On the other hand, 因为 我们有 already
established that the Halting 题目 is

uncomputable, such a program Search cannot exist. 因此, our initial assumption must be wrong, 所以 it isnottruethatarithmeticisbothconsistentandcomplete.
笔记 that in the argument above we rely on the fact that, 给定 a 证明, we can construct a program that mechanistically checks whether it is a valid 证明 of a 给定 命题. This is a manifestation of the intimateconnectionbetweenproofsandcomputation.